

Documentation API Complète – Budget App Backend

Introduction

Cette documentation décrit l'ensemble des endpoints de l'API REST du backend **SpendWiseBack**.

Toutes les routes protégées nécessitent un token JWT dans l'en-tête :

Authorization: Bearer <votre_token>

Authentification

1. Inscription

POST /api/auth/register

Body :

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "123456"
}
```

Réponse :

```
{
  "message": "Utilisateur créé avec succès",
  "user": {
    "id": 1,
    "name": "John Doe",
    "email": "john@example.com"
  }
}
```

2. Connexion

POST /api/auth/login

Body :

```
{
  "email": "john@example.com",
}
```

```
"password": "123456"
}
```

Réponse :

```
{
  "token": "JWT_TOKEN",
  "user": {
    "id": 1,
    "name": "John Doe",
    "email": "john@example.com"
  }
}
```

Utilisateurs

3. Infos utilisateur connecté

GET /api/users/me

Headers : Authorization: Bearer <token>

Réponse :

```
{
  "id": 1,
  "name": "John Doe",
  "email": "john@example.com"
}
```

Transactions

4. Créer une transaction

POST /api/transactions

Headers : Authorization: Bearer <token>

Body :

```
{
  "amount": 100,
  "type": "income", // ou "expense"
  "description": "Salaire",
  "date": "2025-06-14",
  "categoryId": 1
}
```

Réponse :

```
{
  "id": 1,
  "amount": 100,
  "type": "income",
  "description": "Salaire",
  "date": "2025-06-14T00:00:00.000Z",
  "categoryId": 1,
  "userId": 1,
  "createdAt": "...",
  "updatedAt": "..."
}
```

5. Lister les transactions**GET** /api/transactions**Headers :** Authorization: Bearer <token>**Réponse :**

```
[
  {
    "id": 1,
    "amount": 100,
    "type": "income",
    "description": "Salaire",
    "date": "2025-06-14T00:00:00.000Z",
    "categoryId": 1,
    "userId": 1,
    "createdAt": "...",
    "updatedAt": "..."
  },
  ...
]
```

6. Modifier une transaction**PUT** /api/transactions/:id**Headers :** Authorization: Bearer <token>**Body :**

```
{
  "amount": 120,
  "description": "Salaire ajusté"
}
```

Réponse :

```
{
  "message": "Transaction mise à jour",
  "transaction": { ... }
}
```

7. Supprimer une transaction

DELETE /api/transactions/:id**Headers :** Authorization: Bearer <token>**Réponse :**

```
{
  "message": "Transaction supprimée"
}
```

Objectifs (Goals)

8. Créer un objectif

POST /api/goals**Headers :** Authorization: Bearer <token>**Body :**

```
{
  "title": "Économiser pour un voyage",
  "amount": 500
}
```

Réponse :

```
{
  "id": 1,
  "title": "Économiser pour un voyage",
}
```

```
"amount": 500,  
"userId": 1,  
"createdAt": "...",  
"updatedAt": "..."  
}
```

9. Lister les objectifs

GET /api/goals

Headers : Authorization: Bearer <token>

Réponse :

```
[  
  {  
    "id": 1,  
    "title": "Économiser pour un voyage",  
    "amount": 500,  
    "userId": 1,  
    "createdAt": "...",  
    "updatedAt": "..."  
  },  
  ...  
]
```

10. Modifier un objectif

PUT /api/goals/:id

Headers : Authorization: Bearer <token>

Body :

```
{  
  "title": "Voyage à Paris",  
  "amount": 600  
}
```

Réponse :

```
{  
  "message": "Objectif mis à jour",  
  "goal": { ... }  
}
```

11. Supprimer un objectif

DELETE /api/goals/:id

Headers : Authorization: Bearer <token>

Réponse :

```
{
  "message": "Objectif supprimé"
}
```

Catégories

12. Créer une catégorie

POST /api/categories

Headers : Authorization: Bearer <token>

Body :

```
{
  "name": "Alimentation"
}
```

Réponse :

```
{
  "id": 1,
  "name": "Alimentation",
  "createdAt": "...",
  "updatedAt": "..."
}
```

13. Lister les catégories

GET /api/categories

Headers : Authorization: Bearer <token>

Réponse :

```
[
  {
    "id": 1,
    "name": "Alimentation",
```

```
    "createdAt": "...",  
    "updatedAt": "..."  
  },  
  ...  
]
```

Statistiques

14. Statistiques générales

GET `/api/stats`

Headers : `Authorization: Bearer <token>`

Réponse :

```
{  
  "totalIncome": 2000,  
  "totalExpense": 1500,  
  "balance": 500,  
  "transactionsByCategory": [  
    { "category": "Alimentation", "total": 300 },  
    { "category": "Transport", "total": 200 }  
  ]  
}
```

Gestion des erreurs

- **401 Unauthorized** : Token manquant ou invalide
- **404 Not Found** : Ressource non trouvée
- **400 Bad Request** : Données invalides
- **500 Internal Server Error** : Erreur serveur

Sécurité

- Toutes les routes (sauf `/register` et `/login`) nécessitent un JWT valide.
- Les mots de passe sont hashés.
- Les entrées sont validées côté serveur.

Remarques

- Toutes les dates sont au format ISO 8601.
 - Les IDs sont des entiers auto-incrémentés.
 - Les réponses d'erreur sont au format JSON.
-

Pour toute question ou suggestion, ouvrez une issue sur le dépôt